IoT MALWARE DETECTION USING A LAYERED ARCHITECTURE AND MACHINE LEARNING

1st Viet Nhat Nguyen
Faculty of Information Technology
Hanoi University of Civil Engineering
Hanoi, Vietnam
nhatnv@huce.edu.vn

2nd Thi Hong Le Nguyen
Faculty of Information Technology
Hanoi Architectural University
Hanoi, Vietnam
lenth@hau.edu.vn

3rd Le Anh Tuan Nguyen
Faculty of Information Technology
Hanoi University of Civil Engineering
Hanoi, Vietnam
tuan.nlaforwork@gmail.com

4th Duc Huy Nguyen
Faculty of Information Technology
Hanoi University of Civil Engineering
Hanoi, Vietnam
Huy.nd4work@gmail.com

***Abstract*—With the rapid growth of IoT systems, detecting cyberattacks under severe class imbalance remains challenging, as traditional IDS often miss low-frequency stealthy attacks. We propose a layered architecture that decomposes classification into multiple stages, combining LightGBM and XGBoost with tier-specific feature engineering to improve detection of both dominant and minority classes. On the CICIoT2023 dataset, our method achieves a peak F1 Score of 94.25% at 10,000 samples per class and 89.50% at 100,000, outperforming TabNet, NODE, Random Forest and standalone XGBoost/LightGBM. With a low inference latency of 0.0019 ms per sample, the system is suitable for real-time IoT deployment. These findings suggest that the proposed architecture is effective in mitigating class imbalance while improving the detection of minority attack patterns.**

Index Terms—Internet of Things, Cybersecurity, Machine Learning, Layered Architecture.

I. INTRODUCTION

This paper addresses the challenge of severe class imbalance and minority attack detection in IoT intrusion detection systems. Existing flat classification approaches often fail to accurately detect low-frequency stealthy attacks. To overcome this limitation, we propose a layered architecture that decomposes the classification process into sequential stages. Unlike traditional approaches, our method introduces specialized feature sets tailored for each tier and demonstrates improved detection performance for minority classes such as Web-based and Reconnaissance attacks. The main contributions of the research are as follows:

- **Layered Architecture for Classification:** We propose a multi-layered architecture combining LightGBM and XGBoost, tailored for IoT malware detection. This systematically addresses the severe class imbalance inherent in the dataset. Unlike flat models where dominant types overshadow minority classes [3], our approach decouples the process into distinct logical tiers. By isolating categories based on attack behavior, we ensure robust predictive performance and high accuracy for each specific family, regardless of sample size.
- **Comprehensive Exploitation of CICIoT2023:** We maximize the utility of the CICIoT2023 dataset [1] by addressing two significant gaps in literature. First, we detail direct feature extraction from raw PCAP files, a process previous studies have largely bypassed. Second, we explicitly evaluate minority attack families like Web-based and Brute Force attacks. Unlike prior works that often discard these categories to focus on dominant malware [2], our analysis incorporates them for a more realistic and comprehensive IoT threat assessment.
- **Low-Latency Inference for Real-Time Detection:** We validate computational efficiency through fine-grained latency analysis. The system achieves an average algorithmic latency of 0.0019 ms per sample with a throughput exceeding 520,000 samples per second. End-to-end latency remains bounded between $0.0034 - 0.0036$ ms for shallow volumetric paths (DDoS, DoS, Mirai) and $0.0146 - 0.0148$ ms for deep Web and Reconnaissance paths. Runtime decomposition shows inter-tier disk I/O, rather than the model, dominates offline evaluation costs—establishing a foundation for streaming inference on edge devices.

II. RELATED WORK

A. An Overview of IoT Malware

According to [4], IoT's rapid growth is inherently accompanied by surging cybersecurity attacks. Modern IoT devices span diverse CPU architectures and typically operate on resource-constrained, Unix-based platforms. Deployed with weak security configurations, they become attractive targets for Distributed Denial-of-Service (DDoS), DoS, and Brute Force attacks. Furthermore, reliance on vulnerable open-source software accelerates threat proliferation, exemplified by the Mirai botnet's critical risks to various

hardware. Additionally, insecure GPS systems are highly susceptible to Spoofing attacks [6], [7]. Across networks, IoT systems frequently face Reconnaissance (Recon) operations [4], [5], [7] and Web-based probing techniques targeting open ports [4], [7].

B. The Dataset

Legacy benchmarks like NSL-KDD [13] and BotIoT [14] fail to capture modern IoT threat complexity, relying on outdated protocols and lacking diverse device topologies. Consequently, we adopt the CICIoT2023 [1] dataset for three advantages. First, it reflects contemporary vulnerabilities, including modern Mirai variants and slow-rate attacks. Second, its unparalleled taxonomic granularity (33 specific attack classes) is essential for our multi-stage architecture to isolate minority attacks like Web-based and Spoofing. Finally, CICIoT2023 provides raw PCAP files, enabling direct feature extraction to bridge research with real-world traffic monitoring. Anchoring our methodology here ensures our framework remains resilient and applicable to actual deployments.

C. Layered Architecture and Machine Learning Approaches

Existing malware classification primarily relies on flat models that prioritize dominant classes while discarding minority samples. For instance, in [2], Hierarchical Feature Selection focuses on high-volume attacks, omitting critical features like Header_Length—a key metric for identifying subtle anomalies such as Web and Reconnaissance. This inductive bias allows models like MLP [3] to report artificially high F1-Scores (up to 97%) by excluding minority classes, whereas methods like AdaBoost [10] achieve only 78%. To address this, we propose a layered architecture integrating LightGBM and XGBoost. Our approach achieves a 94.25% F1-Score (at 10,000 samples per class) while ensuring minority families are classified accurately rather than overshadowed by dominant classes.

III. Methodology

Fig1 outlines the proposed framework across three phases: data preparation, multi-tier classification, and sub-label prediction. This method efficiently isolates high-volume anomalies, benign traffic, and stealthy attacks within the CICIoT2023 dataset.

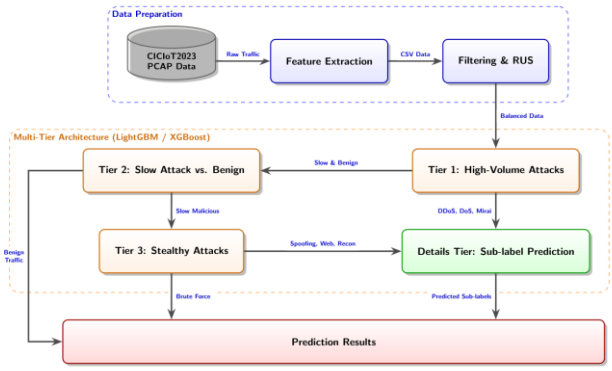


Fig. 1. Overall workflow and multi-tier architecture of the proposed malware classification system

A. Tier Design in the Layered Architecture

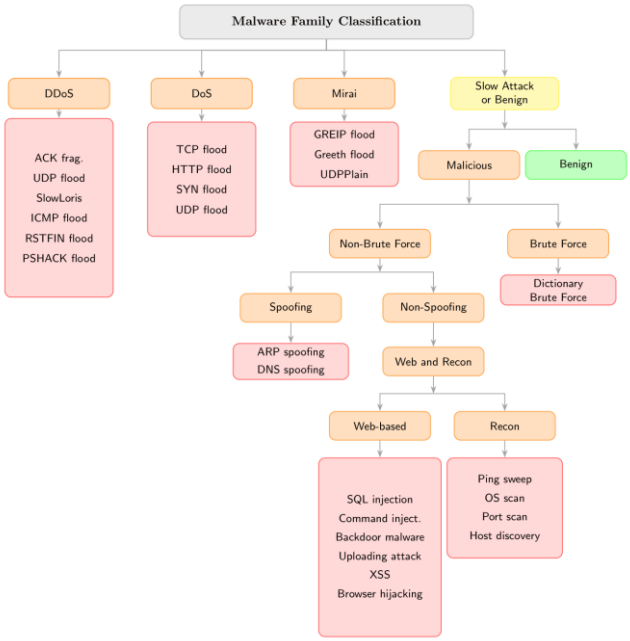


Fig. 2. Malware family classification hierarchy in the CICIoT2023 dataset.

A three-tier main structure plus a dedicated Details Tier balances classification granularity and cumulative error propagation. A flat 34-class model is infeasible due to CICIoT2023's 26,000:1 imbalance ratio and the mutually antagonistic feature regimes of volumetric versus slow-rate attacks (e.g., IAT/Rate vs. Header_Length/Covariance), preventing joint optimization. A two-tier configuration is also insufficient since slow-rate families (Brute Force, Spoofing, Web-based, Reconnaissance) partially overlap with benign traffic [1]. Forcing a single boundary to resolve both malicious-vs-benign and inter-family separations suppresses minority recall. Conversely, exceeding three layers yields diminishing returns by compounding irreversible error propagation and amplifying confusion between Web-based and Reconnaissance attacks, which share near-identical TCP handshakes [1]. Thus, this design provides the minimal required depth: Tier 1 categorizes broad traffic groups, Tier 2 isolates high-volume attacks and establishes the malicious-vs-benign boundary for slow attacks, and Tier 3 categorizes slow-rate families. The Details Tier remains separate for intra-family sub-label discrimination, utilizing leaf-specific feature subsets.

Following [2], this architecture maximizes dataset utility for label-specific classification. Specifically, our layered approach (Fig. 2) is structured as follows:

- **Tier 1:** At the first tier, we categorize traffic into DDoS, DoS, Mirai, and a combined "Slow Attack or Benign" group. Unlike volumetric flooding attacks (DDoS, DoS, Mirai) that exhaust bandwidth [4], [7], slow attacks (Recon-naissance, Brute Force, Web-based, Spoofing) transmit minimal data to evade IDS/IPS detection. Because these low-intensity patterns closely resemble normal behavior, we aggregate them into a unified group [5]–[8].
- **Tier 2:** Here, we focus on the "Slow Attack or Benign" branch to establish a definitive boundary

between malicious and benign traffic. We defer this binary classification to the second tier due to the data scarcity and stealthy nature of slow-rate attacks. By filtering out dominant, high-volume anomalies (DDoS, DoS, Mirai) in Tier 1, the model eliminates overwhelming traffic noise, dedicating its computational focus entirely to isolating subtle malicious behaviors.

- **Tier 3:** Following the isolation of malicious slow-rate traffic, we employ a sequential branching strategy based on fundamental network behaviors. The model identifies the Brute Force family (Tier 3.1) and Spoofing attacks (Tier 3.2). The remaining traffic is grouped into a "Web and Recon" node (Tier 3.3) to compensate for their severe underrepresentation in the dataset [2].
- **Details Tier:** This final tier predicts specific sub-labels within each identified family. By isolating data into distinct family boundaries, the model focuses on nuanced differences between closely related sub-labels, avoiding confusion from overlapping features across malware types and maximizing classification precision.

This layered architecture effectively addresses the severe class imbalance highlighted in [3]. Accurate family-level classification at the initial tier establishes a reliable baseline for multi-stage analysis and prediction.

B. Overview of LightGBM and XGBoost

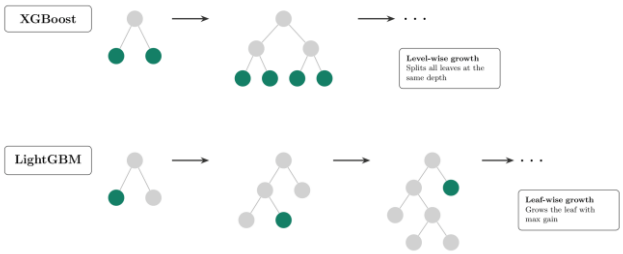


Fig. 3. Illustration about XGBoost and LightGBM

1) XGBoost: Following [9], we integrate eX-treme Gradient Boosting (XGBoost) at deeper nodes to address stealthy attacks. XGBoost employs a level-wise tree growth strategy (Fig. 3), selected for three core advantages: Loss Function Optimization, Overfitting Prevention via Regularization, and Sparsity-Aware Split Finding. We utilize XGBoost at Tier 3.1 and Tier 3.3 to classify:

- Brute Force Detection: XGBoost's advanced optimization excels at capturing subtle fluctuations in packet lengths and inter-arrival times, characteristic of repetitive authentication attempts targeting a singular port [4], [7].

- Reconnaissance and Web-based Attacks: To classify these severely imbalanced, sample-scarce anomalies [4], [5], [7], XGBoost strictly controls tree depth and applies leaf-weight regularization, mitigating inductive bias toward the majority class to ensure robust performance.

2) LightGBM: As detailed in [12], LightGBM is a highly efficient GBDT framework. Unlike XGBoost, it utilizes a leaf-wise strategy growing the leaf with maximum delta loss (Fig. 3). Its superior performance stems from four innovations: Leafwise Tree Growth, Histogram-based Algorithm, Gradient-based One-Side Sampling (GOSS), and Exclusive Feature Bundling (EFB). LightGBM is applied in Tiers 1, 2, and 3.2.

C. K-Fold Cross Validation

Following [2], we employed Stratified K-Fold Cross-Validation to ensure robust, unbiased evaluation under severe class imbalance, preserving per-class sample proportions across folds. To prevent data leakage, samples were split at the row level after random sampling, ensuring no identical records appeared in both training and validation folds. At each tier, the post-RUS pool (Table 2) was partitioned into 5 stratified folds (80/20 train/validation per fold); the 2,382,160-sample test set (Table 3) is drawn from CICIoT2023 partitions disjoint from the training pool, preserving the native class distribution for realistic evaluation.

D. Feature Selection

A significant gap in existing studies [2], [3], [10], [11] is the direct utilization of raw PCAP files. Since machine learning requires structured data, we adopted the University of New Brunswick's extraction tool to convert raw CICIoT2023 PCAPs into CSVs. Our real-world deployment mirrors this pipeline: live traffic is captured via Wireshark, saved as PCAP, and processed through the same tool, ensuring feature consistency (62 initial features) between training and inference. Features were normalized via Standard Scaling based on official benchmarks [1].

- **General Feature Set (Set 1):** Optimized for Tier 1, 2 and final leaf nodes.
- **Specialized Feature Set (Set 2):** Tailored for complex behaviors in Tier 3.1, 3.2, and 3.3

Our analysis shows our proposed sets and the 23-feature baseline in [2] share a core of high impact indicators. These include IAT (the most critical metric), syn_flag_number, Tot sum, Protocol Type, flow_duration, Rate, Std, and protocol-specific markers (e.g., UDP, TCP, ICMP, HTTP, HTTPS, DNS).

Despite these similarities, significant discrepancies emerged, suggesting [2]'s feature set is in-compatible with a multi-level architecture. While [2] dismissed Header_Length as having minimal impact, our results consistently proved it crucial for classification accuracy. We attribute this to [2]'s narrow focus on high-volume attacks (DoS, DDoS, Mirai), overlooking features essential for detecting subtle malware. Consequently, while leveraging their baseline, we prioritized empirical data to finalize two feature sets optimized for our hierarchical system.

Functional Grouping of Selected Features:

- **TCP State and Connection Intent** (syn_flag_number, psh_flag_number, fin_flag_number, rst_flag_number, ack_flag_number): These flags reveal session intents. Excessive syn_flag_number indicates SYN-Flood attacks, while frequent rst_flag_number

signals unauthorized port scanning. Monitoring these identifies protocol-based exploitation.

- **Temporal Dynamics and Traffic Flow** (IAT, flow_duration, Duration, Rate): These metrics characterize transmission pacing. IAT and Rate are critical for detecting volumetric threats (e.g., DDoS), since malicious scripts exhibit higher, rigid frequencies than stochastic legitimate IoT traffic.

- **Protocol and Service Classification** (Protocol Type, TCP, UDP, HTTP, HTTPS, DNS, ICMP): Identifying channels isolates service-specific anomalies. This distinguishes standard traffic from malicious activities like Command and Control (C2) communications over DNS or network reconnaissance via ICMP probes.

- **Statistical Distribution and Payload Profiles** (Tot sum, Std, Variance, Number, Magnitude, Header_Length, AVG, Min, Max): These describe physical packet properties. Variance and Std pinpoint rigid malware payloads, while Header_Length and Magnitude reveal packet complexity, identifying embedded scripts or malformed data.

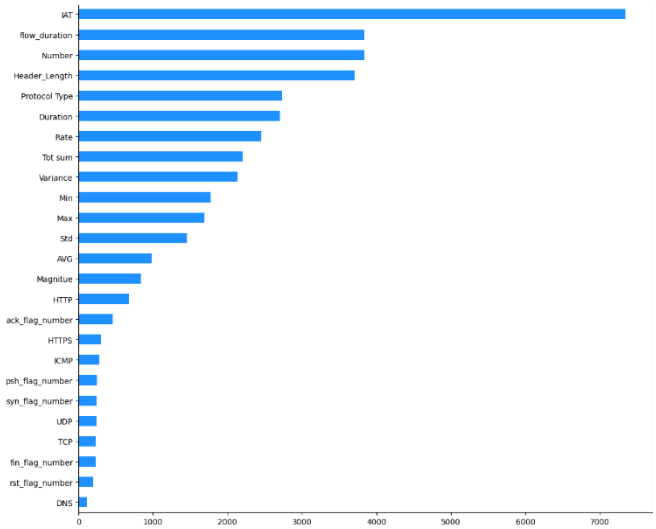


Fig. 4. Feature Importance Ranking of Set 1 at Tier 1

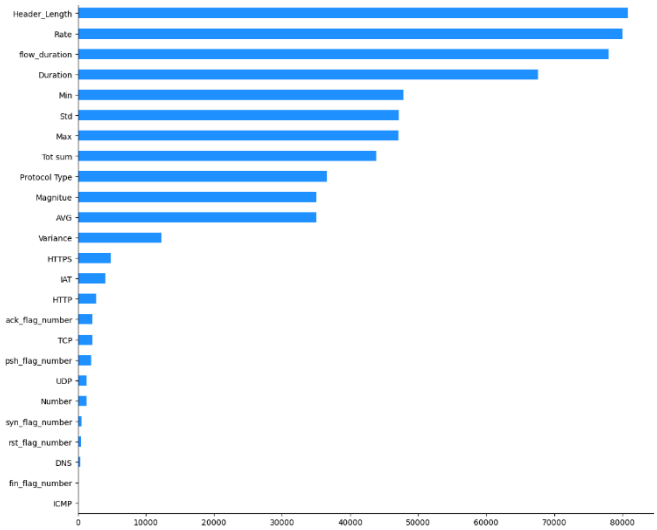


Fig. 5. Feature Importance Ranking of Set 1 at Tier 2.

Figs 4 and 5 illustrate the LightGBM feature importance rankings, highlighting the specific contribution of each attribute in Set 1 to the model's classification performance at Tier 1 and Tier 2, respectively.

The distinct shift in top-ranking features between these two tiers underscores the dynamic nature of feature relevance, further justifying the necessity of a layered classification approach.

While Set 1 is effective for initial tiers, it lacks the precision needed at deeper levels (Slow Attacks Tiers 2, 3.1, and 3.2) to distinguish near-identical patterns like Spoofing versus Recon. Consequently, we introduce Set 2, replacing broad statistical metrics with targeted behavioral features to minimize noise and enhance sensitivity to complex attack signatures.

Specialized Features for Slow Attack Detection

At deeper tiers, Set 2 introduces specialized features designed to capture the subtle, low-frequency behavioral nuances of Slow Attacks that Set 1 might overlook due to its high-volume variance focus.

- **Flow Geometry (Weight, Radius, Co-variance):** These analyze the long-term cluster structure of flows. A consistently low Radius with high Covariance provides a mathematical signature of automated spoofing, exposing slow-rate evasion tactics that single packet analysis misses.
- **Flag Counters (syn_count, ack_count, rst_count, fin_count):** Binary flags are insufficient for stealthy probes. Tracking the cumulative density of flags over extended periods identifies persistent probing behaviors (e.g., Slowloris) that deliberately stagger packets to evade rate-limiters.
- **Directional Rates (Srate, Drate):** Slow attacks target communication balance rather than total bandwidth. These capture subtle asymmetries; for instance, a disproportionate source rate (Srate) relative to a struggling destination response (Drate) signals session exhaustion in Web-based attacks despite low overall traffic.
- **Payload Volume (Tot size):** Monitoring cumulative Tot size over prolonged sessions detects fragmented malware delivery or slow data exfiltration. Even when timing metrics like IAT

appear normal, an abnormal total byte count exposes unauthorized data movement.

- **Direct Protocol Identification (ARP):** Retaining ARP as a dedicated sensor directly isolates protocol anomalies. This is effective against slow Spoofing, where attackers intermittently impersonate gateways to maintain stealthy man-in-the-middle presence.

For Slow Attacks—where malicious activity is deliberately designed to mimic legitimate, lowvolume traffic—removing certain features from Set 1 was a strategic decision to enhance model sensitivity.

- **Elimination of Protocol Noise (DNS):** As Web-based and Reconnaissance attacks operate at the Transport and Application layers (TCP/UDP), DNS metrics are removed to prevent noise from misleading the model.
- **Payload vs. Pattern Contrast:** The optimized set contrasts volume with structural rhythm. Web-based exploits exhibit large Header_Length and Tot size, while Recon-naissance is captured via Covariance and IAT. Removing redundant metrics like Duration accelerates classification.

Figs 6 and 7 present the feature importance rankings extracted from our experimental process, evaluating the specific contribution of each attribute in Set 2 to the model's classification performance at Tier 3.1 and Tier 3.2, respectively.

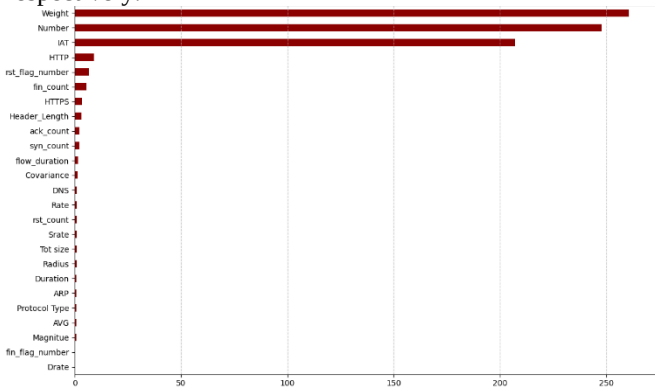


Fig. 6. Eature Importance Ranking of Set 2 at Tier 3.1

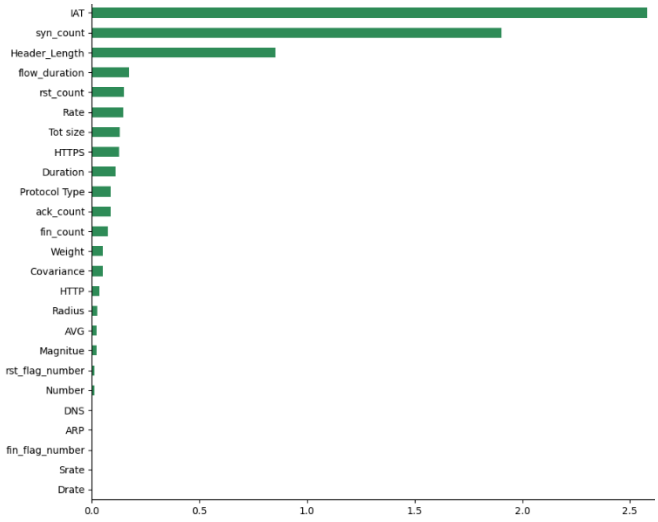


Fig. 7. Feature Importance Ranking of Set 2 at Tier 3.2

To effectively differentiate between Web-based Attacks (e.g., SQLi, XSS) and Reconnaissance (e.g., Port Scanning), the feature set was streamlined to focus on payload complexity and connection patterns. The reduction of the feature set from 25 to 19 attributes is justified by the following technical logic:

Feature Refinement for Web-based vs. Recon

Differentiating Web-based attacks from Recon-naissance is challenging as both share similar TCP handshake patterns. While Reconnaissance targets breadth, Web-based attacks prioritize depth. To resolve this, we derived a specialized subset from Set 2, focusing on structural rhythm and payload characteristics over generic network metrics.

- **Elimination of Protocol Noise (DNS):** Since Web-based and Reconnaissance attacks operate primarily at the Transport and Application layers (TCP/UDP), DNS-specific features are removed. This prevents irrelevant protocol noise from misleading the model during SQL injection or port-scanning identification.
- **Focusing on Connection Phase (Removing FIN flags):** Reconnaissance often results in resets (RST), while Web-based exploits prioritize data exchange (ACK). By removing FIN-related flags, the model focuses on the handshake and transmission phases, where the contrast between rapid probing and complex payloads is sharpest.
- **Payload vs. Pattern Contrast:** The optimized set highlights volume versus structural rhythm. Web-based attacks are identified by large Header_Length and Tot size from malicious scripts, whereas Reconnaissance is captured through Covariance and IAT patterns. Removing redundant timing metrics like Duration ensures a streamlined, faster classification.

Fig 8 visualizes the feature importance ranking of this highly specialized subset, confirming the critical role of payload volume and structural patterns in driving the model's predictions at Tier 3.3.

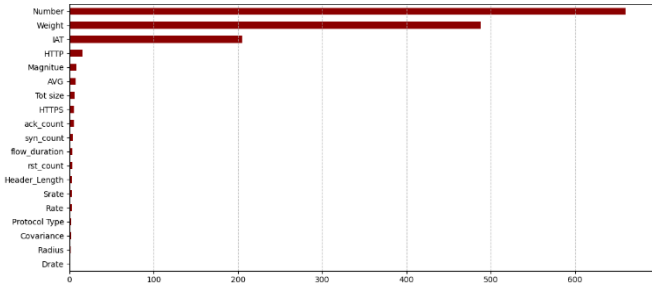


Fig. 8. Feature Importance Ranking of the Streamlined Set for Tier 3.3.

Although a few attributes primarily drive predictions, we retained all features within our optimized subsets. Experiments showed that aggressively pruning features with marginal impact reduced overall accuracy by 2% to 3%, indicating these minor features contribute to complex, non-linear interactions essential for detection.

Furthermore, multi-tier models depend heavily on top-level accuracy, as misclassifications at initial nodes cascade downward, compromising final predictions. Consequently, we prioritized precision over marginal speed gains,

concluding that the slight processing delay is justified by the increase in overall reliability.

These adjustments strike an optimal balance, maintaining architectural efficiency while preserving crucial feature interactions to ensure high sensitivity to complex IoT threats.

IV. Data Processing

A. Data Encoding

In accordance with [10], the categorical labels were numerically encoded (Table 1) to facilitate machine learning processing and optimize memory efficiency. Subsequently, the dataset was filtered, and samples belonging to each label were exported into separate CSV files containing the selected feature sets. This systematic division streamlines the data pipeline for the subsequent multi-tier classification stages.

Table 1. Target Label Encoding Mapping.

ID	Label	ID	Label
0	Backdoor_Malware	17	DictionaryBruteForce
1	BenignTraffic	18	DoS-HTTP_Flood
2	BrowserHijacking	19	DoS-SYN_Flood
3	CommandInjection	20	DoS-TCP_Flood
4	DDoS-ACK_Frag	21	DoS-UDP_Flood
5	DDoS-HTTP_Flood	22	MITM-ArpSpoofing
6	DDoS-ICMP_Flood	23	Mirai-greeth_flood
7	DDoS-ICMP_Frag	24	Mirai-greip_flood
8	DDoS-PSHACK_Flood	25	Mirai-udpplain
9	DDoS-RSTFINFlood	26	Recon-HostDiscovery
10	DDoS-SYN_Flood	27	Recon-OSScan
11	DDoS-SlowLoris	28	Recon-PingSweep
12	DDoS-SynonymousIP	29	Recon-PortScan
13	DDoS-TCP_Flood	30	SqlInjection
14	DDoS-UDP_Flood	31	Uploading_Attack
15	DDoS-UDP_Frag	32	VulnerabilityScan
16	DNS_Spoofing	33	XSS

B. Data Analysis

Table 2. Class Distribution After Random Under-Sampling Strategy.

Label	Count	Label	Count
DDoS-TCP_Flood	100,000	Mirai-greip_flood	100,000
DDoS-SYN_Flood	100,000	Mirai-greeth_flood	100,000
DDoS-PSHACK_Flood	100,000	Mirai-udpplain	100,000
DDoS-RSTFINFlood	100,000	MITM-ArpSpoofing	100,000
DDoS-SynonymousIP	100,000	DNS_Spoofing	100,000
DDoS-ACK_Frag	100,000	Recon-PingSweep	2,262
DDoS-UDP_Flood	100,000	Recon-OSScan	98,259
DDoS-UDP_Frag	100,000	VulnerabilityScan	37,382
DDoS-ICMP_Flood	100,000	Recon-PortScan	82,284
DDoS-ICMP_Frag	100,000	Recon-HostDiscovery	100,000
DDoS-HTTP_Flood	28,790	Backdoor_Malware	3,218
DDoS-SlowLoris	23,426	Uploading_Attack	1,252
DoS-TCP_Flood	100,000	SqlInjection	5,245
DoS-SYN_Flood	100,000	CommandInjection	5,409
DoS-UDP_Flood	100,000	XSS	3,846
DoS-HTTP_Flood	71,864	BrowserHijacking	5,859
BenignTraffic	100,000	DictionaryBruteForce	13,064

To address the severe class imbalance in CICIoT2023 [10], [11], we employed Random Under-Sampling (RUS). Following [10], which demonstrated an accuracy boost from 0.44 to 0.78 using this method, we capped each class at 100,000 instances. As shown in Table 2, majority classes were down-sampled to this threshold, while all minority records were retained to ensure robust evaluation and minimize bias toward dominant threats.

C. Model Training

Following the methodologies in [9], [12], the preprocessed CSV datasets are fed into LightGBM and XGBoost using their standard official APIs to optimize model weights. Finally, the fully trained models are serialized into .pkl files for deployment in the subsequent multi-tier inference stages.

D. Model Evaluation

Table 3: Per-Tier Performance and Wall-Clock Processing Time on 2,382,160 Test Samples.

Classification Tier	Acc.	Prec. (Macro)	Rec. (Macro)	F1 (Macro)	Samples	ms/s
Tier 1: Broad Families	99.98	99.98	99.97	99.98	2,382,160	0.132
Tier 2: Benign vs. Malicious	93.99	99.04	93.95	93.98	658,080	0.359
Tier 2: DDoS Details	99.98	99.95	99.96	99.95	1,052,281	0.539
Tier 2: DoS Details	99.99	99.99	99.99	99.99	371,799	0.514
Tier 2: Mirai Details	99.98	99.98	99.98	99.98	300,000	0.257
Tier 3.1: Brute Force vs. Non-Brute Force	97.77	88.18	98.18	92.91	519,851	0.071
Tier 3.2: Spoofing vs. Non-Spoof.	93.84	93.48	94.35	93.91	489,935	0.059
Tier 3.2: Spoofing Details	92.61	93.38	97.72	92.15	180,189	0.122
Tier 3.3: Web vs. Reconnaissance	93.23	93.32	93.23	93.26	309,746	0.084
Tier 3.3: Web Details	75.92	76.32	72.04	73.66	46,416	2.413
Tier 3.3: Recon. Details	88.37	90.87	83.29	86.06	263,330	1.086
Total Pipeline (incl. aggregation)					2,382,160	0.822

The Brute Force family contains one label (DictionaryBruteForce), requiring no additional classifier. LightGBM used learning_rate=0.05, num_leaves=63, max_bin=255, min_data_in_leaf=50, seed=42, and is_unbalance=True for multi-class tiers. XGBoost utilized learning_rate=0.05 and max_depth=6, tailoring scale_pos_weight to each binary task's ratio to mitigate imbalance. To ensure robustness, 5-fold cross-validation was used per tier, with strict splitting preventing data leakage. The 89.50% aggregate F1-Score masks per-tier variance (99.98% to 73.66%), reflecting specific architectural decisions.

Tier 1 (99.98%). This confirms reliability in identifying volumetric families. Grouping Slow Attack and Benign at Tier 1 spares the classifier from resolving overlapping signatures among Web-Based, Benign, Spoofing, and Reconnaissance traffic [1]. Combined with the inherent majority status and distinct profiles of DDoS, DoS, and Mirai, the high F1-Score results directly from this grouping strategy. Volumetric families isolate cleanly via low IAT, high Rate, and rigid flag distributions, fully captured by Set 1's temporal and counting metrics (Fig. 4). Merging Slow Attacks with Benign yields a wide-margin boundary, limiting cascade contamination to 0.02%.

Tier 2 (99.95–99.99% volumetric details; 93.98% Benign/Malicious). Volumetric categories remain high (99.95–99.99%) due to persistent signatures. Benign/Malicious classification drops to 93.98%, highlighting Slow attacks' subtlety. High Precision (99.04%) paired with lower Recall (93.95%) indicates a strict Malicious threshold, occasionally missing benign-mimicking Slow attacks. Fig. 5 shows Header_Length leading; without floods, signals shift from IAT to header complexity.

Tier 3.1–3.2 (92.91–93.91%). Set 2 substitutes behavioral features for statistical aggregates: Weight (Fig. 6) captures Brute Force's tight clusters, while IAT and syn_count (Fig. 7) expose Spoofing's alternating short sessions. XGBoost outperforms LightGBM in Tier 3.1, driving these results.

Tier 3.3 (93.26%). Pruning to 19 features and removing DNS reduces protocol noise. However, Web/Recon confusion persists as VulnerabilityScan issues HTTP requests with exploit payloads, making them header-equivalent to Web probes. Fig. 8 confirms HTTP loses discriminative power here, making XGBoost appropriate. This 93.26% F1-Score establishes a reliable foundation for subsequent predictions.

Sub-Label Tiers (Web 73.66%, Recon 86.06%). Web's lower F1 stems primarily from severe intra-family imbalance—four of six sub-classes contain under 6,000 samples (Uploading 1,252; Backdoor 3,218; XSS 3,846; SQLi 5,245)—compounded by inaccessible application-layer payloads. The 12-point gap reflects signal location: Web sub-classes share HTTP/HTTPS/TCP handshakes, while Recon sub-classes remain flow-observable (PingSweep/ICMP, PortScan/port-entropy, OSScan/flags). Reverting to Set 1 (Tot sum, ICMP, syn_flag_number) resolves content type better than Set 2's behavioral metrics. This shifting feature importance proves flat models cannot reconcile conflicting distributions.

Table 4. Experimental evaluation results of some model in same setup

Model	Accuracy	Precision (Macro)	Recall (Macro)	F1-Score (Macro)
TabNet	91.86	82.13	75.90	77.20
NODE	90.74	81.28	74.42	75.24
Random Forest	93.99	93.70	77.60	79.72
XGBoost	95.32	88.43	83.04	84.90
LightGBM	94.19	83.27	80.63	81.61
LightGBM & XGBoost (Ours)	95.58	88.09	93.71	89.50

To ensure fair evaluation, all baseline models underwent rigorous hyperparameter tuning on identical data, features, and labels. Each was independently optimized to maximize predictive performance while strictly preventing overfitting and data leakage.

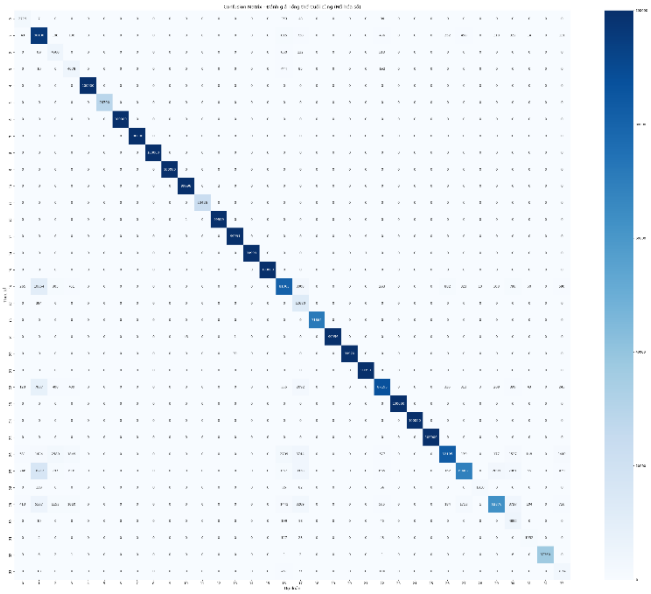


Fig. 9. Confusion Matrix at the 100,000 samples threshold

Master Table Aggregation & Confusion Matrix Analysis. To ensure one prediction per sample, a finalize_prediction routine collapses node trajectories from the deepest activated tier upward. The resulting confusion matrix (Fig. 9) captures these true end-to-end misclassifications, protected by a cross-tier validity filter. At the 100,000-sample threshold, volumetric and Spoofing resolution is near perfect. Off-diagonal errors localize into two distinct regions: intra-Web confusion among payload-sharing sub-classes (SQLi, XSS, Command Injection, Browser Hijacking), and Web/Recon ambiguity driven by VulnerabilityScan's header-equivalent traffic.

Table 5. End-to-End Model Evaluation Across Different Data Thresholds.

Threshold	Accuracy	Precision (Macro)	Recall (Macro)	F1-Score (Macro)
1,000	93.75	93.75	95.03	94.00
10,000	95.02	93.73	95.31	94.25
20,000	95.45	93.73	94.18	93.66
100,000	95.58	88.09	93.71	89.50

Threshold Sensitivity (Table 5). F1 traces 94.00% (1,000) → 94.25% (10,000) → 93.66% (20,000) → 89.50% (100,000). The 10,000 optimum balances training signals against widening imbalance. The F1-score drop to 89.50% at the 100,000-sample threshold reflects the widening imbalance at this scale, as dominant families reach the cap while minorities (e.g., Uploading_Attack with 1,252 samples) remain underrepresented. Sustaining 89.50% under this disparity confirms the robustness of the layered design. This result outperforms TabNet, NODE, Random Forest, and standalone XGBoost/LightGBM by 4.60–14.26 points. This demonstrates that branch-level minority isolation resists the majority-class collapse typical of flat models (Table 4).

E. Computational Efficiency

Table 6. End-to-End Algorithmic Latency per Attack Path.

Attack Path	Tiers	Total Latency (ms)
DDoS	2	0.0036
DoS	2	0.0034
Mirai	2	0.0034
Benign	2	0.0049
Brute Force	3.1	0.0076
Spoofing	3.2	0.0133
Web-based	3.3	0.0146
Reconnaissance	3.3	0.0148

Experiments used Kaggle (Intel Xeon 2-core CPU, 13 GB RAM) with a 2,382,160-sample test set. Table 3 and Table 6 report wall-clock times (I/O plus inference) and end-to-end algorithmic latencies (pure inference), respectively. Per-tier algorithmic latencies are 0.001876, 0.001717, 0.001520, 0.001489, 0.002995, 0.002730, 0.002960, 0.002712, 0.002740, 0.001308, and 0.001514 ms for Tier 1, Tier 2 (DDoS, DoS, Mirai, Benign/Malicious), Tier 3.1, Tier 3.2, Spoofing Details, Tier 3.3, Web, and Recon Details.

The ~ 38-min pipeline (~ 33-min processing) is I/O-bound, making wall-clock time 70–100×higher than average inference (0.0019 ms/sample). Loading a 25-column, 2,382,160-row CSV takes ~ 17.5 s (7.35 µs or 0.0073 ms/row), exceeding single-tier latencies. Thus, wall-clock time reflects batch persistence for reproducibility, not model

limits. Under streaming, latency converges strictly to Table VI values: 0.0034–0.0036 ms for volumetric attacks and 0.0146–0.0148 ms for deep Web and Reconnaissance paths.

V. Conclusion

This paper presented a layered LightGBM-XGBoost architecture for IoT malware classification on the CICIoT2023 dataset. Mitigating class imbalance, it achieves a peak macro F1-Score of 94.25% at 10,000 samples per class (89.50% at 100,000), outperforming TabNet, NODE, Random Forest, and standalone baselines. The framework demonstrates 0.0019 ms average algorithmic latency and end-to-end path latencies from 0.0034–0.0036 to 0.0146–0.0148 ms. However, inter-tier disk I/O dominates the 38-minute runtime, and minority classification (e.g., Web-based attacks, 73.66% F1-Score) lags behind dominant families.

Future work will benchmark PCAP-to-feature extraction and transition to an in-memory streaming engine. Processing from RAM eliminates the 87% disk I/O bottleneck, enabling real-time edge deployment. To mitigate top-down error propagation, we will integrate a confidence-threshold (buffer zone) and a cascade classification strategy. This approach will route low-confidence ("unknown") predictions to a secondary expert model, while also allowing child tiers to identify and intercept overconfident misclassifications—instances where the model assigns a high probability to an incorrect label. Resolving these complex edge cases will further enhance the framework's overall reliability and accuracy.

References

[1] E. C. P. Neto et al., "CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment," Sensors, vol. 23, no. 13, p. 5941, 2023, doi:10.3390/s23135941.

[2] A. Hajjouz and E. Avksentieva, "Optimizing intrusion detection for DoS, DDoS, and Mirai attacks subtypes using hierarchical feature selection and CatBoost on the CICIoT2023 dataset," Data and Metadata, vol. 3, p. 577, 2024, doi: 10.56294/dm2024.577.

[3] H. Q. Gheni and W. L. Al-Yaseen, "Two-step data clustering for improved intrusion detection system using CICIoT2023 dataset," E-Prime - Advances in Electrical Engineering, Electronics and Energy, vol. 9, p. 100673, 2024, doi: 10.1016/j.prime.2024.100673.

[4] Q.-D. Ngo, H.-T. Nguyen, V.-H. Le, and D.-H. Nguyen, "A survey of IoT malware and detection methods based on static features," ICT Express, vol. 6, no. 4, pp. 280–286, 2020, doi: 10.1016/j.icte.2020.04.005.

[5] M. M. Alani and E. Damiani, "XRecon: An Explainable IoT Reconnaissance Attack Detection System Based on Ensemble Learning," Sensors, vol. 23, no. 11, p. 5298, Jun. 2023, doi: 10.3390/s23115298.

[6] M. T. Arafin, D. Anand, and G. Qu, "A Low-Cost GPS Spoofing Detector Design for Internet of Things (IoT) Applications," in Proc. Great Lakes Symp. VLSI (GLSVLSI '17), New York, NY, USA, 2017, pp. 161–166, doi: 10.1145/3060403.3060455.

[7] B. Vignau, R. Khoury, and S. Hallé, "10 Years of IoT Malware: A Feature-Based Taxonomy," in Proc. IEEE 19th Int. Conf. Softw. Qual. Reliabil. Secur. Companion (QRS-C), Sofia, Bulgaria, 2019, pp. 458–465, doi: 10.1109/QRS-C.2019.00088.

[8] M. A. Abbadi, A. M. Al-Bustanji, and M. Al-kasassbeh, "Robust intelligent malware detection using LightGBM algorithm," Int. J. Innov. Technol. Explor. Eng. (IJITEE), vol. 9, no. 6, pp. 1–6, Apr. 2020, doi: 10.35940/ijitee.F4043.049620.

[9] J. Palša et al., "MLMD—A malware-detecting antivirus tool based on the XGBoost machine learning algorithm," Applied Sciences, vol. 12, no. 13, p. 6672, Jul. 2022, doi: 10.3390/app12136672.

[10] S. F. Fahrezi, A. Nugraha, A. Luthfiarta, and N. D. Primadya, "Optimizing Performance of AdaBoost Algorithm through Undersampling and Hyperparameter Tuning on CICIoT 2023 Dataset," Techné: Jurnal Ilmiah Elektroteknika, vol. 23, no. 2, pp. 175–184, 2024, doi: 10.31358/techne.v23i2.467.

[11] S.-M. Tseng, Y.-Q. Wang, and Y.-C. Wang, "Multi-Class Intrusion Detection Based on Transformer for IoT Networks Using CIC-IoT-2023 Dataset," Future Internet, vol. 16, no. 8, p. 284, 2024, doi: 10.3390/fi16080284.

[12] G. Ke et al., "LightGBM: A highly efficient gradient boosting decision tree," in Proc. Adv. Neural Inf. Process.Syst. (NIPS), Long Beach, CA, USA, Dec. 2017, pp. 3146–3154.

[13] M. Tavallaee, E. Bagheri, W. Lu, and A. Ghorbani, "A Detailed Analysis of the KDD CUP 99 Data Set," in Proc. IEEE Symp. Comput. Intell. Secur. Def. Appl. (CISDA), 2009.

[14] N. Koroniotis, "Designing an effective network forensic framework for the investigation of botnets in the Internet of Things," Ph.D. dissertation, Univ. New South Wales, Australia, 2020.